

Task-distribution-weighted observable tangent spaces for repeated multi-angle QAOA: a matched resource audit

Molena Huynh
North Carolina State University
molena.huynh@jmp.com

July 16, 2026

Abstract

Repeated optimization tasks on one graph can differ only in observable weights, suggesting reusable circuit-sensitivity geometry. We define a task-weighted observable tangent space for multi-angle QAOA. If $\mathbf{M} = \mathbb{E}[ww^\top] = LL^\top$, leading left singular vectors of $\mathbf{J}^\top L$ minimize expected discarded local gradient energy. We construct it matrix-free from training weights and evaluate an independent trajectory with matched full- and reduced-space SPSA. Across 16 exact-statevector topology-depth cells from eight graphs, residual-gated reuse changes mean approximation ratio by -1.57 ± 1.04 points (mean ± 2 s.e.) and requires $1.218\times$ the forward evaluations of full SPSA, failing the development gate. Its advantage over a rank-8 random basis is not refresh- or cost-matched. In a separate 256-shot objective pilot on eight graphs, per-task refresh gives $+0.61 \pm 1.57$ points, but construction still uses exact observables and simulator VJPs. Neither pilot demonstrates query, hardware or quantum advantage; together they define a falsifiable representation and its current operational boundary.

The Quantum Approximate Optimization Algorithm (QAOA) alternates problem and mixer unitaries and tunes their angles with a classical optimizer [4, 5]. Multi-angle QAOA (ma-QAOA) replaces the two shared angles per layer with one phase angle per cost term and one mixer angle per vertex [7]. For a graph with $m = |E|$ edges, $n = |V|$ vertices and depth p , this raises the search dimension from $2p$ to

$$d = p(m + n). \quad (1)$$

The additional coordinates can improve expressivity, but only their action on measurable observables can affect an objective. This creates a natural question: can the locally relevant parameter directions be learned once and reused?

The question becomes concrete when one graph supports a sequence of related objectives. Let $F(\boldsymbol{\theta}) \in \mathbb{R}^m$ contain the expected edge-cut observables and let a task be the scalarization $C_w(\boldsymbol{\theta}) = w^\top F(\boldsymbol{\theta})$. Such weight families arise when priorities, scenarios or Pareto scalarizations change while the circuit architecture remains fixed. Multi-objective QAOA makes this repeated-task setting operationally relevant [9]. At any fixed anchor, all task gradients satisfy $\nabla C_w = \mathbf{J}^\top w$, where $\mathbf{J} = \partial F / \partial \boldsymbol{\theta}$. A representation of the range of $\mathbf{J}^\top$ is therefore reusable across weights.

That algebra alone is insufficient. For a single scalar objective, one vector-Jacobian product already returns its complete gradient, while SPSA uses two scalar objective evaluations per iteration in either full or reduced coordinates [13]. Constructing a basis can consequently cost more than direct optimization. A useful study must specify the task distribution, prevent future evaluation weights from entering construction, compare matched optimizers, and report basis, refresh and measurement costs without converting simulator differentiation into fictitious hardware queries.

We develop and audit such a protocol. The representation is weighted by the second moment of a training task distribution, approximated with randomized QB through products with $\mathbf{J}^\top L$ and its transpose. An independent evaluation trajectory tests transfer. Full-space and reduced-space SPSA use

the same 40-step schedule and the same number of scalar objective calls except for an explicitly counted residual gate. Six controls probe reusable geometry, refresh timing and construction cost without isolating all three at matched total cost. We define a development criterion before interpreting the pilot: the gated method must be no more than one approximation-ratio point below full SPSA, exceed a rank-8 random basis with the same maximum rank cap, and use at most $1.25\times$ the full method’s forward evaluations.

The outcome is deliberately not rewritten into a success claim. The learned subspace exceeds a random subspace in an unmatched comparison, but the exact gated method misses the quality condition and refreshes on most opportunities. A small finite-shot pilot is directionally favorable for per-task refresh but is not end-to-end sampled and is statistically unresolved. The contribution is therefore twofold: a theorem and implementation for task-weighted tangent representations, and a matched audit that establishes where the present workflow fails to become an operational advantage.

Contributions.

1. We prove that the leading left singular subspace of $\mathbf{J}^\top L$ is the optimal rank- r parameter representation for expected squared local gradient retention under a specified task-weight second moment.
2. We implement the operator without materializing a dense Jacobian and separate basis-training weights from a disjoint evaluation trajectory that shares only the fixed generative weight model.
3. We conduct a matched full-versus-reduced SPSA audit with no-refresh, fixed-refresh, residual-gated, per-task, random-refresh and random-basis controls, and an additive ledger for scalar objectives, vector observables, shots, sensitivity actions, subspace builds, checks and refreshes.
4. We report the failed exact development gate and the inconclusive hybrid finite-shot pilot as observed, then state a concrete confirmatory and hardware path rather than claiming query or quantum advantage.

Graph symmetries can tie ma-QAOA angles before optimization [12]. Active-subspace analysis instead uses averaged gradient outer products to find influential coordinates [2]. Randomized range finding and fixed-precision QB provide matrix-free tools for approximating such spaces [6, 10, 14]. Noise-aware random-subspace optimizers address a different object: they adapt a classical search subspace and local model under noisy scalar observations rather than factor a vector-observable sensitivity operator [3]. Broad optimizer comparisons also caution that dimension and evaluation count do not rank variational methods by themselves [1].

A graph-conditioned trust-region policy in the first preprint version (now superseded) transfers a pre-query search strategy across graph instances for standard $2p$ -angle QAOA [8]; the present method is instead within-instance and seeks to reuse the $p(m+n)$ -parameter ma-QAOA observable tangent geometry across a family of objective weights. This distinction is the purpose of the cross-reference and is not evidence for the present method.

1 Results

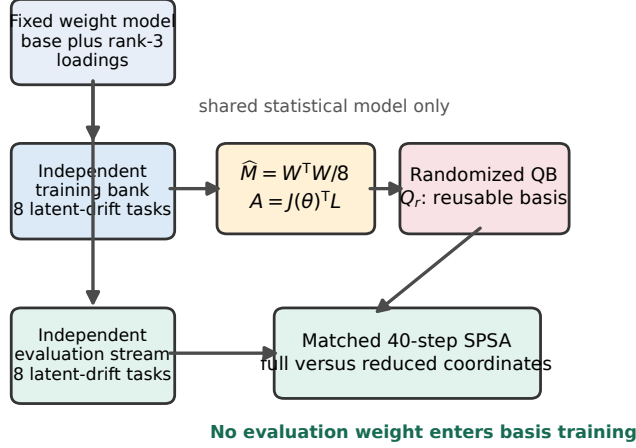
1.1 A task-weighted reusable tangent representation

For an orthonormal basis $Q \in \mathbb{R}^{d \times r}$ and projector $P = QQ^\top$, the task-averaged gradient energy lost outside the basis is

$$\mathbb{E}_w \|\mathbf{I} - P\| \nabla C_w(\boldsymbol{\theta})\|_2^2 = \left\| (\mathbf{I} - P) \mathbf{J}(\boldsymbol{\theta})^\top L \right\|_F^2, \quad \mathbf{M} = \mathbb{E}[w w^\top] = LL^\top. \quad (2)$$

Theorem 1 shows that the leading left singular vectors of $\mathbf{J}^\top L$ minimize this quantity. This is stronger than using the unweighted row space when some observable combinations are common and others are irrelevant to the task distribution. It is also explicitly local: the basis can rotate as $\boldsymbol{\theta}$ changes.

a Leakage-resistant task-stream protocol



b Task-weighted optimality and audit

$$M = \mathbb{E}[ww^T] = LL^T, \quad P_r = Q_r Q_r^T$$

$$\mathbb{E} \|(I - P_r) \nabla C_w\|_2^2 = \|(I - P_r) J^T L\|_F^2$$

Top left singular vectors of $J^T L$ minimize expected discarded local gradient energy.

Development gate: FAILED

Exact gated deficit: -1.57 ± 1.04 points (2 s.e.)

Finite-shot per-task signal: $+0.61 \pm 1.57$ points

The second result is inconclusive and simulator-assisted.

Figure 1: **Task-weighted representation and evaluation boundary.** **a**, A fixed positive-weight model generates independent eight-task training and evaluation trajectories. Only the training bank enters the empirical second moment and matrix-free operator; the learned basis is then used on the evaluation stream. **b**, The task-weighted identity and its rank-constrained solution. The exact development gate failed. The finite-shot per-task result is directionally positive but inconclusive and uses simulator-assisted basis construction.

We estimate $\mathbf{M}$ from eight training weight vectors. Training and evaluation trajectories use independent random seeds and no evaluation weight enters the factor used by randomized QB. They share a fixed per-graph low-rank weight model, which is exactly the assumed transfer relation. Figure 1 shows the separation and the falsification-aware audit.

1.2 The exact development gate fails under a matched SPSA audit

The exact-statevector pilot contains eight distinct graph topologies from random 3-regular and Erdos–Renyi families, $n \in \{8, 10\}$. Evaluating each topology at $p \in \{1, 2\}$ gives 16 configured topology–depth analysis cells; the two depths reuse each topology. Every cell contains eight evaluation tasks, and each method receives 40 SPSA steps per task. Full and reduced SPSA therefore each use 648 scalar objective evaluations over a stream, except that the residual-gated method uses 28 additional scalar evaluations for seven one-probe checks.

Full SPSA obtains mean approximation ratio 0.8244 ± 0.0230 , where uncertainty throughout this section is a descriptive two-standard-error summary over those 16 cells, without modeling their cross-depth dependence. Residual-gated reuse obtains 0.8087 ± 0.0237 . Its paired difference is -1.57 ± 1.04 percentage points. The method therefore misses the prespecified no-worse-than-one-point condition, although it satisfies the forward-cost ceiling and the random-basis comparison. The composite development gate is false.

Per-task recycled refresh is the strongest reduced method in the exact pilot, with mean ratio 0.8136 ± 0.0262 and paired difference -1.08 ± 0.94 points. This is close to, but still below, the one-point development margin. Because the margin and method were selected during development, neither result is a confirmatory equivalence test.

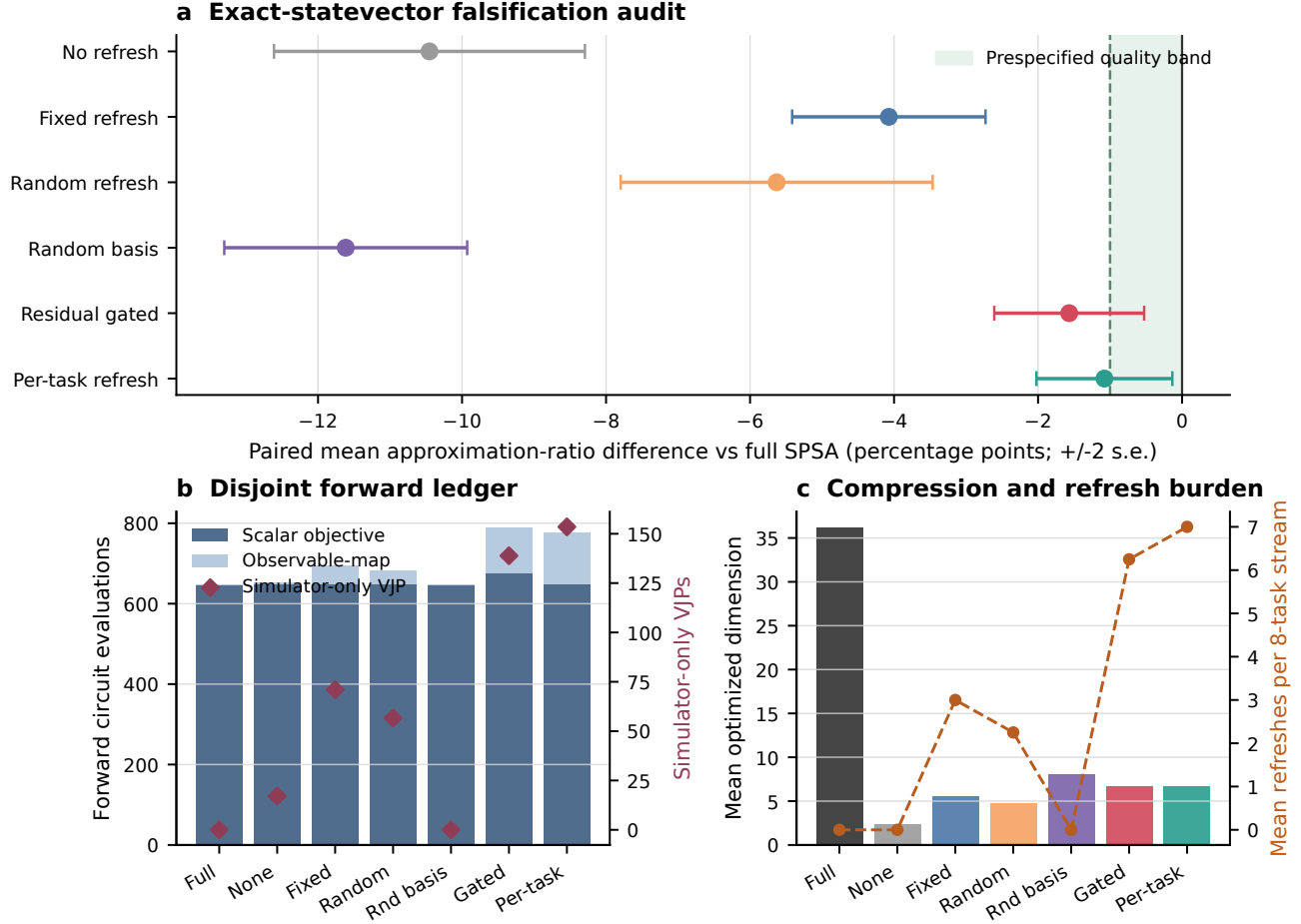


Figure 2: **Exact-statevector quality, cost and dimension audit.** **a**, Paired approximation-ratio differences from full SPSA; error bars are descriptive two-standard-error bars across 16 configured cells. The pale band is the prespecified $[-1, 0]$ -point quality region, not a confidence region. **b**, Mean forward evaluations are partitioned into scalar-objective and vector-observable parameter points; simulator-only VJPs use the right axis and are not added to a hypothetical hardware query count. **c**, Mean optimized dimension and refreshes per eight-task stream.

1.3 Controls are consistent with useful geometry, not operational advantage

The rank-8 random basis loses 11.62 ± 1.69 points relative to full SPSA. Residual-gated reuse exceeds that control by 10.05 points on average. This gap is consistent with useful task-weighted geometry, but the random-basis control is not matched for refresh or construction cost and cannot isolate a causal geometric contribution. A basis built once with no refresh loses 10.45 ± 2.16 points, which is consistent with loss of local-space validity over the warm-started stream. Fixed refresh every two tasks loses 4.07 ± 1.34 points, and a Bernoulli random-refresh schedule loses 5.63 ± 2.17 points. These lower-refresh controls also differ in cost, so they do not isolate refresh timing at a fixed budget.

The residual gate does not demonstrate selective savings. It checks each of the seven task transitions and triggers on 6.25 transitions on average, or 89.3% of opportunities. Per-task refresh updates the recycled basis on all seven transitions. The gated method reduces the mean optimized dimension from 36.19 to 6.61, but uses 676 scalar-objective and 113.25 vector-observable forward evaluations, compared with 648 and zero for full SPSA. Its total forward count is 789.25, or $1.218 \times$ full SPSA, and construction uses a further 138.88 simulator-only VJPs. Per-task refresh uses 776 forward evaluations and 153.50

Table 1: **Exact-statevector development audit.** Ratio summaries are means $\pm$ descriptive two-standard-error summaries over 16 configured cells. Differences are paired within a cell before aggregation. “Obj.” and “Obs.” are disjoint scalar-objective and vector-observable forward parameter points; VJPs are simulator-only reverse-mode actions and are reported separately.

Method	Ratio ± 2 s.e.	Δ full (points)	Dim.	Obj.	Obs.	VJP	Refresh
Full SPSA	0.8244 ± 0.0230	0.00 (ref.)	36.2	648	0.0	0.0	0.00
No refresh	0.7199 ± 0.0238	-10.45 ± 2.16	2.4	648	4.8	17.1	0.00
Fixed refresh	0.7837 ± 0.0236	-4.07 ± 1.34	5.5	648	46.8	71.0	3.00
Residual gated	0.8087 ± 0.0237	-1.57 ± 1.04	6.6	676	113.2	138.9	6.25
Per-task refresh	0.8136 ± 0.0262	-1.08 ± 0.94	6.7	648	128.0	153.5	7.00
Random refresh	0.7681 ± 0.0258	-5.63 ± 2.17	4.7	648	34.8	56.6	2.25
Random basis	0.7083 ± 0.0202	-11.62 ± 1.69	8.0	648	0.0	0.0	0.00

VJPs. These counts establish compression and a useful geometric control; they do not establish fewer queries, lower wall time or hardware deployability.

Figure 3 shows that all structured methods improve as one task warm-starts the next, but the random basis remains far below the learned spaces. The gated refresh rate remains high across the stream. A useful gate would need to predict when refreshing changes downstream performance, not merely produce a large finite-difference residual.

1.4 The finite-shot signal is unresolved and hybrid

We conduct a separate $p = 2$ comparison with 256 computational-basis samples for each scalar objective evaluation, no readout error, and three measurement seeds. The seeds are nested repeats within each of eight graph topologies at $p = 2$ and are averaged before uncertainty is calculated; they are not treated as 24 independent graphs. Full SPSA obtains mean ratio 0.8387 ± 0.0376 . The residual-gated method obtains 0.8353 ± 0.0392 , a paired difference of -0.33 ± 1.67 points. Per-task refresh obtains 0.8448 ± 0.0345 , a paired difference of $+0.61 \pm 1.57$ points. The descriptive uncertainty is too large to motivate a directional claim; no formal directional or equivalence test was performed.

This separate configuration is not a controlled noise ablation of the exact $p = 2$ cells. Initialization and optimizer seeds include the configuration’s job index; removing the $p = 1$ jobs changes that index for later cells. The exact and shot summaries must therefore not be differenced as if they shared identical latent optimizer trajectories.

The pilot is not an end-to-end finite-shot subspace method. Shot sampling is used for SPSA objectives and gated residual calls, whereas every subspace build uses exact-statevector observable evaluations, finite-difference JVPs and reverse-mode VJPs. Full SPSA and per-task refresh each use 165,888 scalar objective shots, but per-task refresh additionally uses 125 vector-observable parameter points and 150.92 simulator-only VJPs on average. Gating uses 173,056 objective shots because its residual checks add 28 scalar calls, plus 95.67 exact vector-observable points and 120.42 simulator-only VJPs. The recorded shot counts are therefore only the scalar-objective component of a hybrid simulator protocol, not a total hardware shot cost.

2 Discussion

The theorem gives a positive structural conclusion: for a specified weight second moment, its leading tangent directions minimize discarded local gradient energy. Empirically, the learned subspace exceeds a random basis with the same maximum rank cap, but that control is not refresh- or cost-matched and

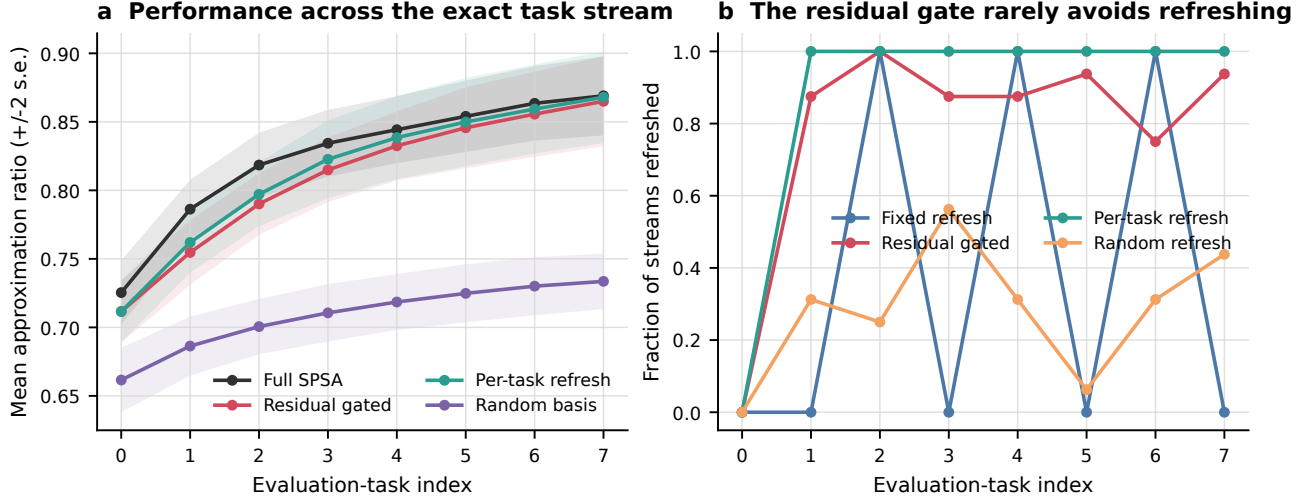


Figure 3: **Task-stream dynamics.** **a**, Exact approximation ratio by evaluation-task index; bands are descriptive two-standard-error summaries across 16 configured cells. The upward trend reflects warm-starting and does not constitute learning across independent graphs. **b**, Fraction of units refreshed at each task. Per-task refresh updates a recycled basis after every transition, the fixed schedule refreshes at tasks 2, 4 and 6, and the residual gate rarely avoids a refresh.

Table 2: **Hybrid 256-shot pilot.** Three measurement seeds are nested within each of eight graph topologies at $p = 2$. Ratios and paired differences are reported as mean $\pm$ two standard errors across those eight units. Shots cover scalar objective and gate calls only; observable-map calls and VJPs used for basis construction remain exact and simulator-only.

Method	Ratio ± 2 s.e.	Δ full (points)	Forward	Shots	VJP	Refresh
Full SPSA	0.8387 ± 0.0376	0.00 (ref.)	648.0	165888	0.0	0.00
Residual gated	0.8353 ± 0.0392	-0.33 ± 1.67	771.7	173056	120.4	5.58
Per-task refresh	0.8448 ± 0.0345	$+0.61 \pm 1.57$	773.0	165888	150.9	7.00

hence does not causally isolate the geometry. The representation also reduces the number of optimized coordinates by more than four fifths on this grid. Neither statement is an operational advantage by itself.

The matched audit rejects the stronger development hypothesis. Full SPSA has the same two-evaluation-per-step property as reduced SPSA, and it avoids basis construction. Gating adds objective calls and suppresses few rebuilds. The exact quality deficit exceeds the prespecified one-point margin. Per-task refresh comes closest to the full baseline, which suggests that local-space rotation, rather than randomized QB accuracy alone, is the main obstacle. It also incurs the largest reverse-mode burden.

The finite-shot pilot does not reverse this conclusion. Its per-task mean is directionally above full SPSA, but eight graph topologies do not resolve the effect and the basis is still supplied by exact simulator sensitivities. In particular, equal scalar-objective shots do not imply equal total quantum cost. All commuting MaxCut edge observables can be estimated from common bitstrings, but the number of shots required for a vector-observable JVP depends on the precision target, noise, compilation and estimator design. Assigning a hardware conversion to an automatic-differentiation VJP would fabricate a cost model, so we leave it explicitly unconverted.

A decisive next study should be locked before new outcomes are inspected. A concrete confirmatory and hardware path is:

1. **Freeze the estimand.** Use the paired difference in mean approximation ratio across topology–

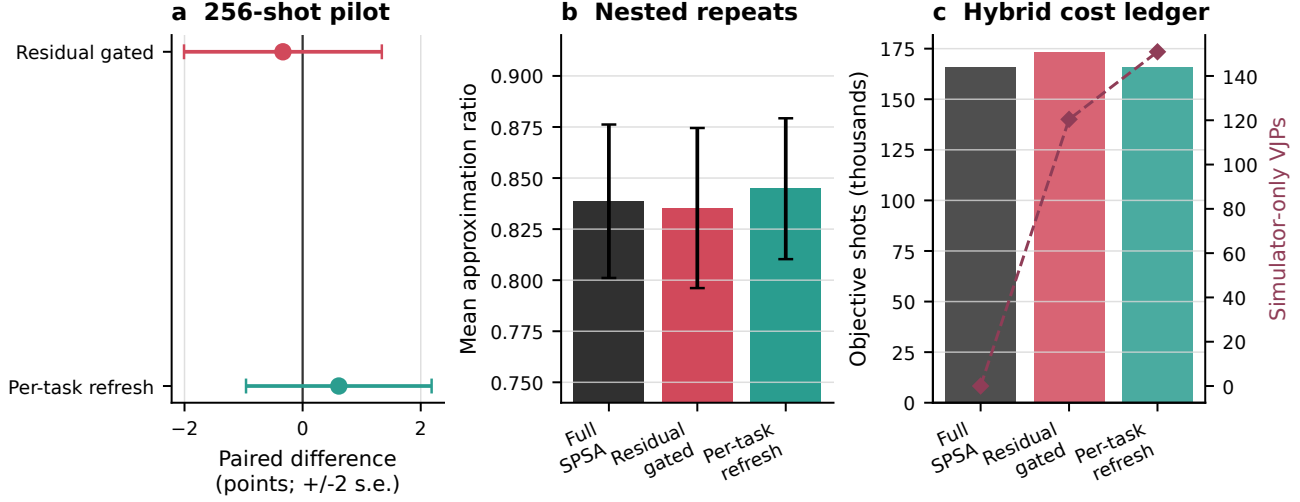


Figure 4: **Hybrid 256-shot development pilot.** **a**, Paired differences from full SPSA with descriptive two-standard-error bars over eight graph topologies. **b**, Mean approximation ratios after averaging three measurement repeats within each topology. **c**, Scalar-objective shots and simulator-only VJPs. The latter have no assigned hardware-shot equivalent, so the bars do not close an end-to-end hardware resource ledger.

depth units, with a one-point equivalence margin. Treat total physical shots and QPU time as co-primary cost outcomes. The observed exact paired standard deviation for gated reuse is 2.08 points; under a Gaussian zero-effect planning approximation, about 35 independent units provide 80% power against a one-point margin. A final power analysis should add attrition, family strata and multiplicity before locking the sample.

2. **Test amortization directly.** Evaluate task-bank sizes $K \in \{1, 2, 4, 8, 16, 32\}$ with training and evaluation banks generated or collected independently. Report the break-even task count using all construction, refresh and optimization resources.
3. **Make construction hardware-native.** Replace simulator VJPs with a specified forward-only or parameter-shift estimator; allocate shots to every scalar and vector-observable circuit point; and record compiled two-qubit gates, circuit batches, queue time and wall time. The current hybrid pilot is not a substitute for this step.
4. **Retain matched controls.** Compare full-space SPSA, no refresh, fixed refresh, gated refresh, random refresh, per-task recycled refresh and a rank-8 random basis under the same maximum rank cap at matched optimizer and total-resource budgets. Add a gate-oracle analysis that asks whether each refresh improves the subsequent task, then calibrate the residual against that event.
5. **Stress the assumed task model.** Include unseen graph families, larger n and p , full-rank or changepoint weight streams, device noise and at least one real backend. Report failures by stratum rather than averaging them away.

Passing this protocol would support an amortized hardware-efficiency claim. The present experiments do not.

3 Methods

3.1 Multi-angle QAOA and vector observables

For an undirected graph $G = (V, E)$ and edge $e = (i, j)$, define $C_e = (I - Z_i Z_j)/2$. A depth- p ma-QAOA state is

$$|\psi(\boldsymbol{\theta})\rangle = \prod_{\ell=1}^p \left[\prod_{i \in V} e^{-i\beta_{\ell i} X_i} \prod_{e \in E} e^{-i\gamma_{\ell e} C_e} \right] |+\rangle^{\otimes n}. \quad (3)$$

The parameter vector contains $p(m+n)$ real angles. The observable map and its Jacobian are

$$F(\boldsymbol{\theta}) = (\langle \psi(\boldsymbol{\theta}) | C_e | \psi(\boldsymbol{\theta}) \rangle)_{e \in E}, \quad \mathbf{J}(\boldsymbol{\theta}) = \frac{\partial F}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{m \times d}. \quad (4)$$

For positive edge weights w , the objective is $C_w(\boldsymbol{\theta}) = w^\top F(\boldsymbol{\theta})$ and $\nabla C_w = \mathbf{J}^\top w$ by the chain rule.

3.2 Task-weighted optimality theorem

Theorem 1 (Optimal task-weighted local parameter subspace). *Let $w \in \mathbb{R}^m$ be square-integrable with second moment $\mathbf{M} = \mathbb{E}[ww^\top] = LL^\top$. Fix a differentiable anchor $\boldsymbol{\theta}$ with Jacobian $\mathbf{J} \in \mathbb{R}^{m \times d}$ and set $A = \mathbf{J}^\top L$. For any $Q \in \mathbb{R}^{d \times r}$ with $Q^\top Q = I$ and $P = QQ^\top$,*

$$\mathbb{E} \|(\mathbf{I} - P)\nabla C_w(\boldsymbol{\theta})\|_2^2 = \|(\mathbf{I} - P)A\|_F^2. \quad (5)$$

If $A = U\Sigma V^\top$, the projector $P_r = U_r U_r^\top$ onto its leading r left singular vectors minimizes Eq. (5) over all rank- r orthogonal projectors, with minimum $\sum_{j>r} \sigma_j(A)^2$.

Proof. Because $\nabla C_w = \mathbf{J}^\top w$ and $\mathbf{I} - P$ is symmetric and idempotent,

$$\begin{aligned} \mathbb{E} \|(\mathbf{I} - P)\mathbf{J}^\top w\|_2^2 &= \text{tr}[(\mathbf{I} - P)\mathbf{J}^\top \mathbf{M} \mathbf{J}(\mathbf{I} - P)] \\ &= \text{tr}[(\mathbf{I} - P)AA^\top(\mathbf{I} - P)] = \|(\mathbf{I} - P)A\|_F^2. \end{aligned}$$

The Eckart–Young–Mirsky theorem, or equivalently the Ky Fan maximum principle applied to $\text{tr}(PAA^\top)$, gives the optimal rank- r projector and the discarded singular-value sum. $\square$

For a training bank $\mathbf{W} \in \mathbb{R}^{K \times m}$ whose rows are task weights, the implementation uses

$$\widehat{\mathbf{M}} = \frac{1}{K} \mathbf{W}^\top \mathbf{W} = LL^\top, \quad L = \frac{1}{\sqrt{K}} \mathbf{W}^\top. \quad (6)$$

The theorem then holds exactly for the empirical training distribution. It does not guarantee transfer to a different evaluation distribution, global optimization convergence, or preservation after the anchor moves.

3.3 Matrix-free construction and objective-conditioned refresh

Randomized QB accesses $A = \mathbf{J}^\top L$ through

$$Av = \mathbf{J}^\top(Lv), \quad A^\top u = L^\top(\mathbf{J}u), \quad (7)$$

using a simulator VJP for the first action and a central-finite-difference JVP for the second. The development configuration uses maximum rank 8, block size 2, relative tolerance 0.10, six norm probes and four residual probes. A build can stop at tolerance or at the available rank; its status, convergence

flag and residual are persisted in the result CSV. A refresh seeds QB with the current basis but permits the resulting span to rotate.

For task w_t at current θ , the gate estimates the relative omitted objective-gradient energy. For Gaussian direction v it computes central differences along v and $(\mathbf{I} - P)v$. With s probes,

$$\hat{\rho}^2 = \frac{\sum_{j=1}^s [D_{(\mathbf{I}-P)v_j} C_{w_t}(\theta)]^2}{\sum_{j=1}^s [D_{v_j} C_{w_t}(\theta)]^2}. \quad (8)$$

Each probe requires four scalar objective evaluations. Its numerator and denominator separately estimate squared gradient norms under exact directional derivatives, but their ratio is not unbiased and one probe is not a calibrated confidence bound. The exact pilot uses $s = 1$, finite-difference step 0.01 and refresh threshold 0.30. The finite-shot pilot uses step 0.10 to reduce noise amplification. The gate is therefore an operational heuristic, not a theorem.

The refresh controls are: one initial build only (no refresh); rebuild at tasks 2, 4 and 6 (fixed); check and rebuild above threshold at tasks 1–7 (gated); refresh a recycled basis for every task (per-task); independent refresh probability 0.35 at each transition (random); and one isotropic Gaussian orthonormal basis of maximum rank 8 with no sensitivity construction (random basis).

3.4 Matched full and reduced SPSA

For either native parameters or reduced coordinates z with $\theta = \theta_0 + Qz$, SPSA uses a Rademacher vector δ_k and

$$\hat{g}_k = \frac{C(x_k + c_k \delta_k) - C(x_k - c_k \delta_k)}{2c_k} \delta_k, \quad (9)$$

where x_k denotes θ_k or z_k . We use 40 steps, $a_k = 0.12/(k + 1 + 5)^{0.602}$, $c_k = 0.10/(k + 1)^{0.101}$, one perturbation average, and maximum update norm 0.40. One final evaluation gives $2 \times 40 + 1 = 81$ scalar objective calls per task and 648 per eight-task stream. The highest observed candidate becomes the warm start for the next task. Full and reduced methods share the same initial angle and seed schedule, although their perturbation vectors live in different dimensions.

3.5 Weight streams and graph design

For each graph, a seeded model draws a positive base vector and an orthonormal loading matrix of latent rank 3. A latent AR(1) process with persistence 0.85 and innovation scale 0.18 perturbs the weights through those loadings. Values are clipped below at 0.10 and each task vector is normalized to mean one, so objective scale cannot masquerade as optimizer improvement. The training and evaluation streams use different seeds but the same base and loading model. This is a controlled synthetic transfer setting, not a claim about naturally occurring task distributions.

The exact grid contains random 3-regular and Erdos–Renyi graphs with edge probability 0.5, $n \in \{8, 10\}$, graph seeds 41 and 42, and $p \in \{1, 2\}$. It contains eight graph topologies and 16 configured topology–depth analysis cells; depths reuse topology. The finite-shot grid retains $p = 2$, hence eight graph topologies and eight cells. Brute-force weighted MaxCut supplies the denominator of each approximation ratio. A complex-128 PyTorch statevector backend supplies the exact observable map [11].

3.6 Exact and shot evaluators

The exact evaluator returns statevector expectations. The shot evaluator samples computational-basis bitstrings from the same statevector and computes all commuting edge observables from each bitstring. The pilot uses 256 shots per scalar objective call and zero readout error. Measurement and SPSA seeds are separate. Candidate selection is based on the sampled objective, but the reported primary approximation

ratio is the exact statevector expectation at the selected candidate. This post-evaluation makes optimizer comparisons less noisy but further distinguishes the pilot from a hardware-only workflow.

Subspace construction does not call the shot evaluator. It uses exact observable-map calls, finite-difference JVPs and automatic-differentiation VJPs. Thus “256-shot pilot” describes its scalar objective path only.

3.7 Resource ledger

Every task emits cumulative and task-local counts for:

- scalar objective parameter points;
- vector-observable parameter points;
- computational-basis shots used by the evaluator;
- sensitivity JVPs and simulator-only VJPs;
- subspace builds, residual checks and triggered refreshes.

Scalar-objective and vector-observable points are non-overlapping, and their sum is reported as forward circuit evaluations. JVP is an operation-level counter whose underlying finite-difference points are already represented by vector-observable evaluations; it is not added again. A VJP is recorded in a separate simulator-only category because no physical circuit conversion is assumed. Builds, checks and refreshes are events rather than circuit costs. This ledger prevents double counting while preserving implementation detail.

3.8 Measurement reuse and an amortization break-even condition

For MaxCut, every edge observable is diagonal in the computational basis. A single bitstring batch at a fixed parameter vector therefore estimates the entire vector $F(\boldsymbol{\theta})$, and every scalarization in a task bank can be computed from that same batch. For independent and identically distributed computational-basis shots, let $\hat{F}_B$ be the empirical edge-cut vector from B bitstrings. Then, for any finite collection of weights $w_1, \dots, w_K$,

$$\hat{C}_{w_k} = w_k^\top \hat{F}_B, \quad \text{Cov}(\hat{C}_{w_j}, \hat{C}_{w_k}) = \frac{1}{B} w_j^\top \Sigma_F w_k, \quad (10)$$

where Σ_F is the single-bitstring covariance of the edge indicators. Thus evaluating K scalarizations at the *same* circuit point costs one measurement batch, not K independent batches. This reuse does not extend across different parameter vectors: once task optimizers follow different trajectories, their circuit calls cannot be merged by scalarization alone.

This distinction yields a transparent break-even calculation for a future hardware-native coordinate-gradient experiment. Suppose $0 \leq r < d$, a full central finite-difference update uses $2d$ circuit points, a rank- r reduced update uses $2r$, each task receives S updates, and basis construction plus all refreshes use a cost C_{basis} that is fixed with respect to K in this comparison. Across K tasks the reduced method uses fewer circuit points than the full method exactly when

$$C_{\text{basis}} + 2rSK < 2dSK \quad \Longleftrightarrow \quad K > \frac{C_{\text{basis}}}{2S(d-r)}. \quad (11)$$

If construction or refresh decisions make the basis cost depend on task-bank size, replacing it by $C_{\text{basis}}(K)$ gives an implicit realized-cost condition rather than an explicit threshold. The same inequality holds for shots when every circuit point receives the same shot allocation, but equal shots need not give equal directional-gradient precision because full and reduced directions can have different signal and variance.

Equation (11) is an accounting identity, not a novelty or efficiency result: it says nothing about matched solution quality, and it does not apply to SPSA because full and reduced SPSA both use two objective calls per update. The released experiments use SPSA and simulator-assisted subspace construction, so they do not test this break-even opportunity.

3.9 Analysis and development criterion

For each method and topology–depth cell we average approximation ratio across the eight sequential tasks. In the finite-shot pilot, the three measurement repeats are first averaged within the same cell. Paired differences subtract full SPSA in the same cell. We report means and $2s/\sqrt{N}$, where s is the sample standard deviation of cell summaries. In the exact pilot, the 16 cells reuse eight topologies across two depths and their dependence is not modeled. The resulting two-standard-error bars are descriptive, not formal confidence intervals, independent-sample uncertainty estimates or equivalence tests.

The development-only gate requires residual-gated reuse to (i) have paired mean difference at least -0.01 relative to full SPSA, (ii) exceed a random basis with the same maximum rank cap, and (iii) use at most $1.25\times$ the mean forward circuit evaluations of full SPSA. Because hyperparameters and this criterion were examined on the development grid, a fresh locked dataset is required for confirmatory inference.

3.10 Protocol integrity and reproducibility

Configuration, the complete transitive `rsqaoa` Python source tree, the runner, the analyzer and protocol metadata are SHA-256 hashed before execution. The frozen exact and finite-shot protocol hashes begin `86641437f696` and `f17516bb20c6`, respectively; both result files record implementation hash `fe49dff72483`. Randomized-QB build JSON, stop reason, convergence flag and residual are persisted on build rows. Row-level ledgers sum exactly to their final cumulative values. The committed development implementation is preserved at Git commit `b35216d8a3ac`; later physical-accounting additions intentionally change the live source tree and must not be retroactively assigned to these rows. The complete regeneration path is:

```
git checkout b35216d8a3ac
python experiments/freeze_amortized_protocol.py
python experiments/run_amortized.py
python experiments/analyze_amortized.py
python experiments/make_amortized_paper_assets.py
python -m pytest tests -q
python run.py validate
```

The figure/table script does not rerun experiments. It validates CSV/JSON row counts and protocol hashes, nests measurement repeats, and writes a manifest containing source and display hashes.

A separate confirmatory design is recorded in `experiments/configs/nmi_confirmatory_design.yaml`. It declares 40 new graph-topology units, independent training and evaluation weights, a K sweep from 1 to 32 tasks, changepoint and independent-weight stress tests, finite-shot/readout perturbations, target-hitting costs and gate calibration against oracle next-task refresh benefit. Required controls include full SPSA, per-task refresh, a mean-weight rank-one basis, an unweighted observable-Jacobian basis, a dense-SVD oracle, symmetry-tied ma-QAOA, an empirical gradient-bank active subspace and physical forward-coordinate methods. Retained-gradient energy and principal-angle drift are recorded alongside optimization gain. It is a design artifact, not an external preregistration: its implementation is incomplete and execution is explicitly disabled until a registration receipt is recorded. It contributes no result to this manuscript and makes the next decision boundary auditable rather than retrofitted.

3.11 Use of language-model assistance

A generative language model was used during software implementation, analysis support and manuscript drafting. The author reviewed the resulting code, recomputed the numerical and statistical outputs from the released data, verified the scientific statements and assumes full responsibility for the work.

4 Limitations

The exact pilot has eight graph topologies, two small sizes, depths one and two, two graph families and a synthetic rank-3 task model. It is a development grid, not a locked test. Its descriptive error bars use 16 configured topology–depth cells without modeling the reuse of each topology across depth. The task second moment is estimated from only eight training weights. Each topology–depth cell has one seeded training bank and one evaluation trajectory; shot repeats vary measurements, not task streams. Generalization over stream realizations is therefore not separately identified from graph variation. Hyperparameters are not independently tuned for each control. Warm-started tasks are dependent within a stream.

The residual estimate uses one Gaussian probe and is not calibrated. The randomized-QB stop status and residual are persisted, but its four-probe residual indicator is randomized rather than a confidence certificate. The randomized basis has maximum rank 8, and quality may change under larger ranks, different observable dictionaries or dense-oracle construction. The theorem is local and makes no statement about SPSA convergence, finite-difference bias or basis rotation. The exact simulator stops at $n = 10$ and does not test large-scale statevector, tensor-network or QPU execution.

The finite-shot pilot has eight graph topologies and no device noise. Its configuration-dependent job indices change initialization and optimizer seeds relative to the exact grid, so it is not a controlled noise comparison. Only scalar objective calls are sampled; subspace construction and primary post-evaluation remain exact-statevector operations. Its shot count is therefore incomplete as a hardware ledger. No claim is made about quantum advantage, QPU speed, wall-clock reduction, asymptotic scaling, Pareto-front quality or deployment.

5 Data and code availability

No external dataset is used. The accompanying [GitHub repository](#) contains the row-level exact and finite-shot CSV files, machine-readable summaries, frozen protocols, configurations, plotting/table script, generated display manifest, source package and tests. Code is released under the MIT License. The current repository release has no archival DOI; a versioned deposit should accompany any formal journal submission.

6 Author contributions

M.H. conceived the study, developed the method and software, designed and ran the experiments, analyzed and visualized the results, and wrote and revised the manuscript.

7 Competing interests

The author declares no competing interests.

8 Correspondence

Correspondence should be addressed to Molena Huynh (molena.huynh@jmp.com).

9 Conclusion

Task-distribution-weighted observable tangent spaces give a precise local answer to which ma-QAOA directions matter across a family of scalarized objectives. The matched audit gives an equally precise empirical boundary. On the present exact grid, the learned subspace exceeds a random basis in an unmatched comparison but remains below full SPSA and costs more forward evaluations; the residual gate rarely avoids refreshing. The directionally positive finite-shot per-task result is unresolved and depends on exact simulator sensitivities. The correct conclusion is therefore not query efficiency, but a reusable representation whose operational case remains unproven. The released controls, ledger and confirmatory path make that case falsifiable.

References

- [1] Xavier Bonet-Monroig, Hao Wang, Diederick Vermetten, Bruno Senjean, Charles Moussa, Thomas Bäck, Vedran Dunjko, and Thomas E. O’Brien. Performance comparison of optimization methods on variational quantum algorithms. *Physical Review A*, 107:032407, 2023. doi: 10.1103/PhysRevA.107.032407.
- [2] Paul G. Constantine, Eric Dow, and Qiqi Wang. Active subspace methods in theory and practice: Applications to kriging surfaces. *SIAM Journal on Scientific Computing*, 36(4):A1500–A1524, 2014. doi: 10.1137/130916138.
- [3] Kwassi Joseph Dzahini, Jeffrey M. Larson, Matt Menickelly, and Stefan M. Wild. A noise-aware scalable subspace classical optimizer for the quantum approximate optimization algorithm. In *2025 IEEE International Conference on Quantum Computing and Engineering*, pages 20–30, 2025. doi: 10.1109/QCE65121.2025.00013.
- [4] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014. doi: 10.48550/arXiv.1411.4028.
- [5] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019. doi: 10.3390/a12020034.
- [6] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806.
- [7] Rebekah Herrman, Phillip C. Lotshaw, James Ostrowski, Travis S. Humble, and George Siopsis. Multi-angle quantum approximate optimization algorithm. *Scientific Reports*, 12:6781, 2022. doi: 10.1038/s41598-022-10555-8.
- [8] Molena Huynh. Query-efficient quantum approximate optimization via graph-conditioned trust regions, 2026. arXiv:2604.24803v1.
- [9] Ayse Kotil, Elijah Pelofske, Stephanie Riedmüller, Daniel J. Egger, Stephan Eidenbenz, Thorsten Koch, and Stefan Woerner. Quantum approximate multi-objective optimization. *Nature Computational Science*, 5:1168–1177, 2025. doi: 10.1038/s43588-025-00873-y.
- [10] Per-Gunnar Martinsson and Joel A. Tropp. Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29:403–572, 2020. doi: 10.1017/S0962492920000021.
- [11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [12] Kaiyan Shi, Rebekah Herrman, Ruslan Shaydulin, Shouvanik Chakrabarti, Marco Pistoia, and Jeffrey Larson. Multi-angle QAOA does not always need all its angles. In *2022 IEEE/ACM 7th Symposium on Edge Computing*, pages 414–419, 2022. doi: 10.1109/SEC54971.2022.00062.

- [13] James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi: 10.1109/9.119632.
- [14] Wenjian Yu, Yu Gu, and Yaohang Li. Efficient randomized algorithms for the fixed-precision low-rank matrix approximation. *SIAM Journal on Matrix Analysis and Applications*, 39(3):1339–1359, 2018. doi: 10.1137/17M1141977.